

# Analyseur de langage clair — aide-mémoire

**En une phrase** : une application Flask qui reçoit un texte administratif en français et **montre dans le texte** ce qui l'éloigne des dix principes de rédaction claire de la Commission européenne. Chaque signalement a une position exacte, un principe et, si possible, une reformulation. **Pas de note sur 100** : un score fait optimiser un chiffre, un surlignage fait relire.

**3**règles : longueur (P4),  
connecteurs (P5), passif  
(P8)**48**

tests + 1 xfail, en ~3 s

**4/6**passif sur les phrases  
de référence (2/6  
dépendances seules)**25**mots : seuil de phrase  
en français (21 en  
anglais)**16 · 17**connecteurs lourds ·  
verbes avec *être***4**formats : .txt .md .docx  
.odt (.pdf .doc refusés)

## DÉROULÉ DES 21 DIAPOS — LE MESSAGE DE CHACUNE

|       |                                                                                 |
|-------|---------------------------------------------------------------------------------|
| 1     | Titre : laisser la phrase à l'écran, on y revient à la fin.                     |
| 2     | Qui je suis : 1 min 15 max, finir sur le lien avec le sujet.                    |
| 3     | Problème : correct ≠ lisible (44 mots, 6 locutions).                            |
| 4     | Sujet : l'outil montre l'endroit du problème.                                   |
| 5–6   | Référentiel : 3 principes détectés ; 2 pistes ; 5 hors de portée d'une machine. |
| 7     | Public : agents publics, juristes. Pas de score : décision assumée.             |
| 8     | <b>Démonstration</b> du parcours complet (≈ 5 min).                             |
| 9     | Stack : citer 2 motifs (pysbd, spaCy), pas le tableau.                          |
| 10    | Chaîne : un seul point d'entrée, <code>build_document()</code> .                |
| 11    | Architecture : un dossier, une responsabilité ; deux registres.                 |
| 12    | <b>Règle d'or</b> : <code>texte[start:end] == fragment</code> .                 |
| 13–15 | Normaliser une fois ; segmenter 285 + 49 ; tokeniser 285 + 36.                  |
| 16    | Passif : même forme, quatre constructions ; une regex échoue.                   |
| 17–18 | spaCy : l'arbre de dépendances, puis 5 questions posées à l'arbre.              |
| 19    | Heuristiques ① cop ② VERB ③ verbes avec <i>être</i> : 2/6 → 4/6.                |
| 20    | Bilan : fait, à faire, refusé ; gel le 23/09.                                   |
| 21    | Fin : « Pour assurer le suivi, le service publie une notice. »                  |

## COMMANDES

```
.venv\Scripts\Activate.ps1
flask db upgrade      # après chaque migration
flask seed            # lexiques.json → base ; relançable
flask run              # http://127.0.0.1:5000
.venv\Scripts\python.exe -m pytest -p no:cacheprovider
# une seule suite, en direct :
... -m pytest -p no:cacheprovider tests/test_passif.py -v
```

Lanceur cassé → `.venv\Scripts\python.exe -m flask run`. Script d'activation refusé → `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`. **Ne pas recréer le venv** : `requirements.txt` n'épingle pas les versions.

## DÉMONSTRATION — CHECK-LIST

- ☐ `.venv\Scripts\Activate.ps1` puis `flask run`
- ☐ Base à jour : `flask db upgrade` **puis** `flask seed`
- ☐ Une analyse à blanc : spaCy se charge (~2 s)
- ☐ « à cet égard » **absent** de la liste (sinon le supprimer)
- ☐ Dossier `exemples/` ouvert, un `.pdf` sous la main
- ☐ Terminal prêt pour `pytest` ; zoom navigateur 125 %

### Le parcours (diapo 8)

**1 Analyser** les deux paragraphes : les trois règles se déclenchent ; longueur et connecteurs se superposent sans `<mark>` imbriqués.

**2 Cliquer un surlignage** : la fiche s'ouvre (principe, position, proposition).

**3 Listes de mots** : coller « À cet égard, le dossier est complet. » → rien. Ajouter « à cet égard » → « sur ce point ». Relancer → signalé. Aucun redémarrage.

**4 Enregistrer** sous un nom → redirection → *Analyses enregistrées* → rouvrir : surlignage identique. La base ne stocke **aucun HTML**.

**5 Importer** notification.docx (mêmes signalements que les 3 autres formats), puis un **.pdf** : message dédié, pas « format inconnu ».

### Le texte de démo : docs/soutenance.md, § 6

```
§ 1 la phrase de 44 mots # P4 + P5 superposés
§ 2 « une notice est publiée » # sans agent → avertissement
    « est rejetée par le biais..» # avec agent → info
```

**Plan B** : la capture de la diapo 8, puis `pytest tests/test_passif.py -v`. Vérifié le 26/09 : 12 connecteurs, 1 phrase longue, 2 passifs.

## GARDE-FOUS ET RÉGLAGES

|                      |                                                                                  |
|----------------------|----------------------------------------------------------------------------------|
| MAX_TEXT_LENGTH      | 20 000 caractères, vérifié côté serveur                                          |
| MAX_CONTENT_LENGTH   | 2 Mo ; au-delà : 413, même page, message dédié                                   |
| MAX_FORM_MEMORY_SIZE | aligné sur 2 Mo : un texte trop long reçoit son propre message                   |
| NOM_MAX              | 100 caractères pour le nom d'une analyse                                         |
| Modèle spaCy         | <code>fr_core_news_sm 3.8.0</code> , épinglé par URL                             |
| Réseau               | aucune requête à l'exécution ; listes dans <code>data/seeds/lexiques.json</code> |

## ORDRE DE SACRIFICE

conteneurisation → export → anglais → administration → règles supplémentaires → tokenisation fine. **Jamais** : la chaîne, les trois règles, le surlignage, le passif, les tests, la répétition.

## LE CŒUR TECHNIQUE : L'INVARIANT DES POSITIONS

```
document.texte[start:end] == fragment.texte # partout
```

Chaque couche **transporte** sa position par addition, elle ne la recherche jamais :

```
bloc.start() 285 # début du 2e paragraphe
+ match.start() 36 # « est » dans la phrase
= token.start 321 # « est publiée » = 321 → 332
```

**Les deux pièges** : `texte.find()` retombe sur la 1<sup>re</sup> occurrence d'une phrase répétée ; cumuler les longueurs dérive de 2 caractères par ligne vide.

**XSS** : `surligner()` échappe **chaque segment** puis insère les `<mark>`.  
Jamais l'inverse : on échapperait ses propres balises, et `< → &lt;` ; change les longueurs.

## LA CHAÎNE, DANS L'ORDRE

```
saisie, ou fichier → extraire() # texte brut
build_document(texte_brut, langue="fr")
normalize() # une seule fois
segment() # paragraphes → phrases
tokenize() # \S+ avec offset
analyser_phrases() # spaCy, phrase par phrase
run(document) # REGLES → Finding, triés
surligner() # échappe, puis <mark>
```

## LES TROIS RÈGLES LIVRÉES

| Règle              | Principe                  | Mécanisme                                                                                                         |
|--------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| longueur_phrase    | P4 · avertissement        | compte les tokens de la phrase, compare au seuil de la langue (25)                                                |
| connecteurs_lourds | P5 · info                 | 16 locutions en base, chacune avec sa reformulation ; motif tolérant : <code>\s+</code> , casse, élision « d' »   |
| passif             | P8 · info / avertissement | aux:pass ou cop → participe VERB → pas un verbe avec <i>être</i> → cherche obl:agent ; sans agent = avertissement |

## NOTIONS PYTHON À SAVOIR EXPLIQUER

**@dataclass(frozen=True)** — génère `__init__`, `__repr__`, `__eq__` ;  
frozen interdit la modification : un signalement ne peut pas être déplacé en route.

**Finding vs FindingRecord** — l'un transporte (pur Python, toujours créé), l'autre stocke (ligne SQL, seulement si on enregistre). Mêmes noms de champs : `FindingRecord(**asdict(finding))`. À la relecture, le gabarit lit les deux sans distinction (*duck typing*).

**Rule(ABC) + @abstractmethod** — contrat : impossible d'instancier une règle sans `check()`. Aucun état sur `self` : l'instance est partagée par toutes les requêtes.

**Extracteur = fonction, règle = classe** — l'extracteur n'a rien à porter ; la règle porte id, principe, sévérité, langues.

**Liste vs dictionnaire** — REGLES est parcourue en entier ; REGISTRE est indexé par extension, on en cherche un seul.

**create\_app()** — fabrique d'application : une config de test (base en mémoire) sans toucher la vraie.

**POST-Redirect-GET** — rafraîchir la page ne réenregistre pas l'analyse.

**Idempotence** — `normalize(normalize(t)) == normalize(t)` : c'est ce qui rend la relecture exacte.

**Tuple** — séquence non modifiable, de longueur quelconque ;  
`tuple[str, ...]` = « autant de str qu'on veut ».

### 3 · LES DÉCISIONS DE DESIGN, ET POURQUOI

#### ARCHITECTURE

**Couches qui ne descendent que vers le bas** (web → services → règles → données) — chaque morceau se teste seul : les règles ne connaissent pas Flask, les routes ne font pas de linguistique.

**Fabrique `create_app()`**, extensions branchées par `init_app()` — une application neuve par test, avec `TestConfig` et une base en mémoire.

**Deux blueprints**, `analyze` et `admin` — séparer analyse et administration ; les URL se construisent par leur nom (`url_for`).

**Configuration par variables d'environnement** (`.env`, modèle `.env.example`) — aucun secret dans le code ; `TestConfig` hérite et ne change que deux valeurs.

#### TEXTE ET POSITIONS

**Un seul point d'entrée**, `build_document()`, **qui normalise une fois** — un seul texte de référence : toutes les positions pointent dedans.

**Positions reportées par addition, jamais cherchées** — `find()` se trompe sur une phrase répétée ; cumuler des longueurs dérive à chaque ligne vide.

**`normalize()` idempotent** — relire une analyse redonne le même texte, donc les mêmes positions ; la même fonction nettoie les saisies des listes.

**`pysbd`, `clean=False`, `char_span=True`** — ne coupe pas sur « M. » ou « art. », et rend des positions sans réécrire le texte.

**Tokenisation `\S+`, grossière et assumée** — elle suffit à compter les mots ; l'analyse fine vient de `spaCy`, à part.

**Dataclasses figées** (`frozen=True`) — personne ne peut décaler une position après coup.

#### SPACY

**`linguistics.py`, seule frontière avec `spaCy`**, qui rend des `TokenLinguistique` maison — changer de bibliothèque ne toucherait que ce fichier.

**`spaCy` analyse les phrases de `pysbd`, en un seul lot** — une seule définition de « phrase » dans le projet ; gouverneur reste un index valide ; un lot va plus vite.

**Modèle chargé une fois (@cache), sans ner** — il est lourd, et la reconnaissance d'entités ne sert à rien ici.

**`fr_core_news_sm`, pas md** — comparé sur les phrases de référence : aucun gain.

#### MOTEUR DE RÈGLES

**Règle abstraite + `signaler()`** — sans `check()`, l'erreur tombe au démarrage ; une règle ne peut pas oublier son id ni son principe.

**Règles sans état** — une seule instance sert toutes les requêtes sans mélanger leurs résultats.

**Registre REGLES écrit en clair, décorateur retiré** — tout ce que le moteur exécute se lit d'un coup d'œil, sans import à effet de bord.

**`run()` trie par position** — le surlignage parcourt le texte une seule fois, de gauche à droite.

**Seuils et lexiques hors du code, par langue** ; `.get()` rend `None` — une langue non couverte n'exécute aucune règle, jamais de `KeyError`.

**Connecteurs : regex tolérante sur `document.texte`**, expressions longues d'abord — positions absolues sans calcul, pas de double signallement.

**Passif = dépendances + heuristiques** — mesuré : 2/6 → 4/6.

**Pas de score global** — montrer l'endroit du problème, pas une note arbitraire à optimiser.

#### RENDU HTML

**Échapper chaque segment, puis baliser** — pas d'injection, pas d'entité & ; coupée en deux.

**Découper aux frontières de tous les signalements** — deux signalements qui se chevauchent donnent un seul <mark> à deux classes, un HTML valide.

**`page()`, rendu unique**, gestionnaire 413 compris — surlignage et résumé calculés à un seul endroit ; une page d'erreur reste une page d'analyse.

**JavaScript minimal**, le serveur prépare `data-findings` — sans JS, la page reste complète.

#### IMPORT DE FICHIERS

**Même motif que les règles** : `contrat base.py`, registre, une fonction par format — deux axes d'extension, une seule idée à expliquer.

**REGISTRE est un dictionnaire**, REGLES une liste — on cherche un extracteur par extension, on exécute toutes les règles.

**Un extracteur est une fonction**, une règle une classe — l'extracteur n'a ni état ni attributs à porter.

**Un extracteur ne normalise jamais** — sinon texte saisi et fichier importé auraient deux textes de référence.

**`base.py` n'importe rien du projet** — c'est ce qui casse le cycle `registry` → `docx` → `registry`.

**Erreurs `ExtractionError` au message lisible** ; le `try` n'entoure qu'une ligne — une vraie erreur de programmation n'est pas masquée.

**`.pdf` et `.doc` refusés avec un message dédié** — pas de paragraphes fiables / format binaire obsolète ; mieux qu'un « format inconnu ».

**`.txt` : UTF-8 d'abord, 7 encodages candidats, sinon refus** — éviter les idéogrammes sur un texte court ; refuser vaut mieux qu'analyser du « Å© ».

**`.odt` : générateur récursif** — garder l'ordre de lecture entre titres et paragraphes.

**Le nom de fichier du client ne sert qu'à choisir l'extracteur** — jamais utilisé comme chemin sur le disque.

#### PERSISTANCE

**Finding (transport) ≠ FindingRecord (table), mêmes champs** — moteur testable sans base, traduction en une ligne avec `asdict()`.

**Repository : tout le SQL dans `repositories.py`** — des fonctions au nom métier ; les routes et règles ignorent le stockage.

**« Analyser » n'enregistre rien** ; on enregistre à la demande, sous un nom — pas de base encombrée d'essais.

**On stocke le texte normalisé et les empan, jamais le HTML** — une seule vérité ; un HTML relu comme sûr serait une faille ; les classes CSS ne restent pas figées.

**L'enregistrement relance l'analyse côté serveur** — on ne croit jamais des positions venues du navigateur.

**Listes de mots en base, source versionnée `lexiques.json`, flask seed idempotent** — git suit le JSON ; relancer n'écrase rien ; aucune requête réseau.

**Passage en base sans toucher aux règles** — `connecteurs_lourds()` et `verbes_conjugues_avec_etre()` ont gardé leur signature.

**UniqueConstraint, cascade, version\_tokeniseur** — la base refuse les doublons ; supprimer une analyse supprime ses signalements ; un futur tokeniseur ne cassera pas les anciennes positions.

**Imports sous `TYPE_CHECKING`** — les types sans le cycle `rules` → `lexiques` → `repositories` → `rules`.

#### WEB

**POST-Redirect-GET** après chaque écriture — rafraîchir ne rejoue pas l'enregistrement.

**Retours anticipés dans `index()`** ; le fichier l'emporte sur la zone de texte — code lisible de haut en bas ; déposer un fichier est un choix plus délibéré.

**`MAX_FORM_MEMORY_SIZE` aligné sur 2 Mo** — un texte trop long reçoit le message du projet, pas un 413 brut.

**Suppression par POST, jamais par un lien GET** ; saisie des listes normalisée et en minuscules — pas de suppression accidentelle ; une apostrophe courbe collée correspond quand même.

**Ni CSRF ni authentification** — application locale, mono-utilisateur ; Flask-WTF ajouterait une dépendance.

## QUESTIONS PROBABLES

**Pourquoi pas Flesch ou un indice de lisibilité ?**

Un indice rend un chiffre, pas un endroit. Et il compte des syllabes : peu transposable d'une langue à l'autre.

**Pourquoi une regex pour les connecteurs, spaCy pour le passif ?**

Une locution figée est une suite de mots ; le passif est une structure. Chaque outil là où il est nécessaire.

**Pourquoi spaCy ne découpe-t-il pas les phrases ?**

Mesuré : il fusionne un titre avec la phrase suivante. pysbd découpe, spaCy analyse des phrases déjà isolées.

**Et fr\_core\_news\_md ?**

Comparé sur les phrases de référence : aucun gain. Le petit modèle suffit.

**Un <script> collé dans le texte ?**

Échappé segment par segment, affiché tel quel.

**Un fichier de 50 Mo ?**

Refusé par MAX\_CONTENT\_LENGTH avant la route ; app\_errorhandler(413) rend la même page, via page().

**Pourquoi le formulaire ne renvoie-t-il pas les signalements ?**

On ne se fie jamais à des positions venues du navigateur : le serveur relance l'analyse.

**Pourquoi le JSON et la base ?**

Le JSON est la source versionnée (git ne suit pas la base) ; flask seed reconstruit la base sans rien écraser.

**Un .txt en cp1252 ?**

UTF-8 d'abord, sinon charset-normalizer sur 7 candidats ; rien de sûr → refus plutôt que du mojibake.

**Pourquoi refuser le PDF ?**

Pas de paragraphes fiables : les coupures feraient mentir segmentation, positions et surlignage.

**Ajouter l'anglais ?**

Seuil dans seuils.py, lexique dans le JSON, une ligne dans REGLES pour les règles propres à l'anglais.

## LIMITES ASSUMÉES — À DIRE SOI-MÊME

- « La porte est ouverte », « Il est convaincu » : état ou passif, ambigu → signalés (un xfail le documente).
- Tokenisation \S+ : la ponctuation reste collée au mot. Suffisant pour compter.
- « s'agissant **des** » non détecté : le lexique traite des locutions figées, pas la morphologie.
- Enregistrer relance les règles : si une liste change entre-temps, l'enregistré diffère de l'affiché.
- Base migrée mais non amorcée : rien ne plante, mais les connecteurs disparaissent. D'où flask seed.
- Pas de CSRF ni d'authentification : application locale, mono-utilisateur (Flask-WTF = une dépendance de plus).
- Tableaux d'un .docx ignorés ; extracteur choisi sur l'extension, pas les octets.
- Date d'enregistrement en UTC : seule la date est affichée.
- Annotation de segment() inexacte : list[list[dict]] au lieu de list[dict]. Python ne la vérifie pas, rien ne plante.

## VOCABULAIRE LINGUISTIQUE

|                  |                                                                                     |
|------------------|-------------------------------------------------------------------------------------|
| <b>empan</b>     | couple (début, fin) de caractères dans le texte normalisé                           |
| <b>lemme</b>     | forme du dictionnaire : <i>prise</i> → <i>prendre</i> , <i>allée</i> → <i>aller</i> |
| <b>aux:pass</b>  | auxiliaire d'un passif (« été » dans « a été prise »)                               |
| <b>cop</b>       | copule ; repli quand spaCy n'étiquette pas aux:pass                                 |
| <b>obl:agent</b> | complément d'agent : « par le conseil »                                             |
| <b>NFC</b>       | forme Unicode composée : « é » = 1 caractère, pas 2                                 |
| <b>POS VERB</b>  | catégorie exigée du participe : écarte « est susceptible »                          |

## LA PHRASE DE CONCLUSION

Avant : Afin d'assurer le suivi, une notice est publiée.

Après : **Pour** assurer le suivi, **le service publie** une notice. — P5 (mot simple) et P8 (agent nommé).

## PASSIF : LES SIX PHRASES DE RÉFÉRENCE

| Phrase                                  | Attendu | Dépendances seules | Règle actuelle |
|-----------------------------------------|---------|--------------------|----------------|
| La décision a été prise par le conseil. | passif  | ✓                  | ✓              |
| La décision a été prise.                | passif  | X                  | ✓ (repli cop)  |
| Le rapport sera publié demain.          | passif  | ✓                  | ✓              |
| Elle est allée à Paris.                 | non     | X                  | ✓ (liste être) |
| La porte est ouverte.                   | ambigu  | X                  | X (xfail)      |
| Il est convaincu de son bon droit.      | non     | X                  | X              |
| <b>Score</b>                            |         | <b>2/6</b>         | <b>4/6</b>     |

Source : revue du code du 23/09, docs/soutenance.md, CLAUDE.md et le code au 26/09/2026. Tests vérifiés le 26/09 : 48 passent, 1 xfail.